

09 - Opencv

Docupedia Export

Author:Lima Queila (CtP/ETS)

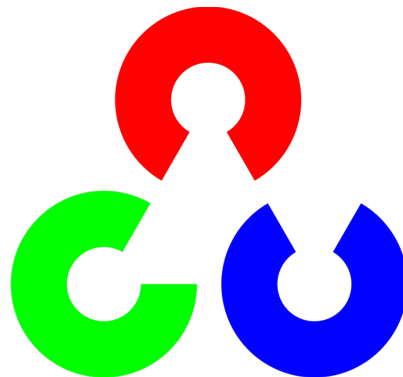
Date:03-Mar-2026 17:05

Table of Contents

1	TREINAMENTO DE PYTHON (OPENCV)	4
2	Personalizando a Janela	8
2.1	LINHAS	8
2.2	RETÂNGULO	9
2.3	CÍRCULO	10
2.4	TEXTO	11
2.5	EXERCÍCIO	12
2.6	Transformação em espaço de cores	14
2.6.1	RGB/BGR:	14
2.6.2	HSV:	16
2.7	Mudança de espaço de cor	16
2.7.1	BGR → HSV	17
2.7.2	BGR → CINZA	18
3	Eventos do Mouse	20
3.1	EXERCÍCIO	21
3.2	EXERCÍCIO	23
4	VÍDEOS	26
4.1	EXERCÍCIO	28
4.2	EXERCÍCIO	30

1

TREINAMENTO DE PYTHON (OPENCV)



Uma biblioteca de código aberto que inclui centenas de algoritmos de visão computacional.

Possui suporte para diversas linguagens de programação como C++, Java e Python.

O OpenCV Python, utiliza o **Numpy**, e converte todas as estruturas de **vetores (arrays)** em **Numpy arrays**.

Isso facilita sua integração com outras bibliotecas como, por exemplo, Matplotlib.

```
import cv2
import numpy as np
```

A função **cv2.imread()** é utilizada para ler uma imagem.

A imagem deve estar salva na pasta em que estamos trabalhando ou o caminho completo dela deve ser fornecido.

- **cv2.IMREAD_COLOR** ou apenas **1**: Carrega a imagem sem transparência caso haja. É o **default**.
- **cv2.IMREAD_GRAYSCALE** ou apenas **0**: Carrega a imagem em tons de cinza.
- **cv2.IMREAD_UNCHANGED** ou apenas **-1**: Carrega a imagem com todos os canais (incluindo o de transparência , caso haja).

```
dog = cv2.imread('dog.jpg', cv2.IMREAD_COLOR)
dog_cinza = cv2.imread('dog.jpg', cv2.IMREAD_GRAYSCALE)
dog_transp = cv2.imread('dog.jpg', cv2.IMREAD_UNCHANGED)
```

cv2.imshow() mostra a imagem em uma janela.

O primeiro argumento é o nome da janela (uma string) e o segundo é a imagem.

cv2.waitKey() & **0xFF** é uma função atrelada ao teclado. Aceita números inteiros como argumento que representam tempo em **milissegundos**. A função será executada por esse tempo especificado.

Se **zero (0)** for passado como argumento, a janela aguarda indefinidamente por uma tecla ser pressionada. DEVE SER USADA para que uma imagem seja mostrada. A função retorna o valor da tecla apertada.

```
cv2.imshow('dog', img)
x = cv2.waitKey(0)
print('TECLA APERTADA:', x)
cv2.destroyAllWindows()
```

Podemos criar quantas janelas quisermos, desde que tenham **nomes diferentes**.

```
cv2.imshow('dogão', dog)
cv2.imshow('dogão2', dog_cinza)
cv2.imshow('dogão3', dog_transp)
cv2.waitKey(0)
```

Para especificar a tecla que deverá ser pressionada, será utilizada a tabela ASCII

VALOR DECIMAL	VALOR HEXA-DECIMAL	CONTROL CARACT.	CARACT.	VALOR DECIMAL	VALOR HEXA-DECIMAL	CARACT.	VALOR DECIMAL	VALOR HEXA-DECIMAL	CARACT.	VALOR DECIMAL	VALOR HEXA-DECIMAL	CARACT.	VALOR DECIMAL	VALOR HEXA-DECIMAL	CARACT.	VALOR DECIMAL	VALOR HEXA-DECIMAL	CARACT.
000	00	NUL		043	2B	+	086	56	V	129	81	ü	172	AC	¼	215	D7	#
001	01	SOH	☺	044	2C	,	087	57	W	130	82	ë	173	AD	í	216	D8	≠
002	02	STX	☺	045	2D	-	088	58	X	131	83	ä	174	AE	«	217	D9	┘
003	03	ETX	♥	046	2E	.	089	59	Y	132	84	å	175	AF	»	218	DA	┘
004	04	EOT	♦	047	2F	/	090	5A	Z	133	85	ä	176	B0	▨	219	DB	■
005	05	ENQ	♣	048	30	0	091	5B	[	134	86	ä	177	B1	▨	220	DC	■
006	06	ACK	♠	049	31	1	092	5C	\	135	87	ä	178	B2	▨	221	DD	■
007	07	BEL	•	050	32	2	093	5D	]	136	88	ä	179	B3		222	DE	■
008	08	BS	◼	051	33	3	094	5E	^	137	89	ä	180	B4	—	223	DF	■
009	09	HT	○	052	34	4	095	5F	_	138	8A	ä	181	B5	—	224	E0	α
010	0A	LF	☉	053	35	5	096	60	`	139	8B	ï	182	B6	—	225	E1	β
011	0B	VT	♂	054	36	6	097	61	a	140	8C	î	183	B7	—	226	E2	γ
012	0C	FF	♀	055	37	7	098	62	b	141	8D	ï	184	B8	—	227	E3	π
013	0D	CR	♫	056	38	8	099	63	c	142	8E	Ë	185	B9	—	228	E4	Σ
014	0E	SO	♫	057	39	9	100	64	d	143	8F	Ë	186	BA		229	E5	σ
015	0F	SI	☼	058	3A	:	101	65	e	144	90	É	187	BB	┘	230	E6	μ
016	10	DLE	▶	059	3B	;	102	66	f	145	91	æ	188	BC	┘	231	E7	τ
017	11	DC1	◀	060	3C	<	103	67	g	146	92	Æ	189	BD	┘	232	E8	φ
018	12	DC2	↑	061	3D	=	104	68	h	147	93	ö	190	BE	┘	233	E9	⊖
019	13	DC3		062	3E	>	105	69	i	148	94	ö	191	BF	┘	234	EA	Ω
020	14	DC4		063	3F	?	106	6A	j	149	95	ö	192	C0	┘	235	EB	δ
021	15	NAK	§	064	40	@	107	6B	k	150	96	û	193	C1	┘	236	EC	∞
022	16	SYN	—	065	41	A	108	6C	l	151	97	ü	194	C2	┘	237	ED	∅
023	17	ETB	┘	066	42	B	109	6D	m	152	98	ÿ	195	C3	┘	238	EE	€
024	18	CAN	┘	067	43	C	110	6E	n	153	99	Û	196	C4	—	239	EF	⌒
025	19	EM	┘	068	44	D	111	6F	o	154	9A	Ü	197	C5	+	240	FO	≡
026	1A	SUB	—	069	45	E	112	70	p	155	9B	ë	198	C6	┘	241	F1	≡
027	1B	ESC	—	070	46	F	113	71	q	156	9C	£	199	C7	┘	242	F2	≡
028	1C	FS	┘	071	47	G	114	72	r	157	9D	¥	200	C8	┘	243	F3	≡
029	1D	GS	↔	072	48	H	115	73	s	158	9E	₣	201	C9	┘	244	F4	↑
030	1E	RS	▲	073	49	I	116	74	t	159	9F	f	202	CA	┘	245	F5	J
031	1F	US	▼	074	4A	J	117	75	u	160	A0	ä	203	CB	┘	246	F6	+
032	20	SP	Space	075	4B	K	118	76	v	161	A1	í	204	CC	┘	247	F7	≈
033	21		!	076	4C	L	119	77	w	162	A2	ö	205	CD	—	248	F8	°
034	22		"	077	4D	M	120	78	x	163	A3	û	206	CE	—	249	F9	•
035	23		#	078	4E	N	121	79	y	164	A4	ñ	207	CF	—	250	FA	•
036	24		\$	079	4F	O	122	7A	z	165	A5	Ñ	208	D0	—	251	FB	√
037	25		%	080	50	P	123	7B	{	166	A6	°	209	D1	—	252	FC	∩
038	26		&	081	51	Q	124	7C		167	A7	°	210	D2	—	253	FD	?
039	27		'	082	52	R	125	7D	}	168	A8	ë	211	D3	—	254	FE	•
040	28		(	083	53	S	126	7E	~	169	A9	—	212	D4	—	255	FF	
041	29		)	084	54	T	127	7F	⌒	170	AA	—	213	D5	—			
042	2A		*	085	55	U	128	80	Ç	171	AB	½	214	D6	—			

cv2.destroyAllWindows() destrói todas as janelas criadas.

Para destruir apenas uma janela específica, use a função **cv2.destroyWindow()** e passe o nome exato da janela a ser destuída como argumento.

Usamos a função **cv2.imwrite()** para salvar uma imagem. O primeiro argumento será o nome do arquivo e o segundo a imagem que desejamos salvar.

```
cv2.destroyAllWindows  
cv2.imwrite('dog_cinza.jpg', dog_cinza)
```

2 Personalizando a Janela

ALGUNS TERMOS COMUNS QUE SERÃO UTILIZADOS:

- **Image:** A imagem onde desenharemos as formas.
- **Color:** Cor das formas. Para **BGR** (Blue, Green, Red), passamos uma tupla de 3 valores (X,Y,Z). Para tons de **cinza**, passamos apenas um valor (X).
- **Thickness:** Espessura da linha, Default = 1. Se for o valor -1, a figura é preenchida.
- **LineType:** Tipo de linha. **cv2.LINE_AA** é ideal para curvas.

2.1 LINHAS

cv2.line() é usado para desenhar uma linha, com coordenadas de início e fim da linha. Vamos criar uma imagem em preto e desenhar 4 linhas brancas, duas verticais e duas horizontais.

1º argumento: imagem a ser usada.

2º argumento: coordenada x,y de início. **3º argumento:** coordenadas de x,y de fim.

4º argumento: cores BGR.

5º argumento: espessura

```
import cv2
import numpy as np

# Criando uma imagem preta de tamanho 512x512
img = np.zeros((512, 512, 3), np.uint8)

# Linhas Horizontais (imagem, (xi,yi), (xf,yf), (color), (thickness))
cv2.line(img, (150,0), (150,512), (255,255,255), 5)
cv2.line(img, (350,0), (350,512), (255,255,255), 5)

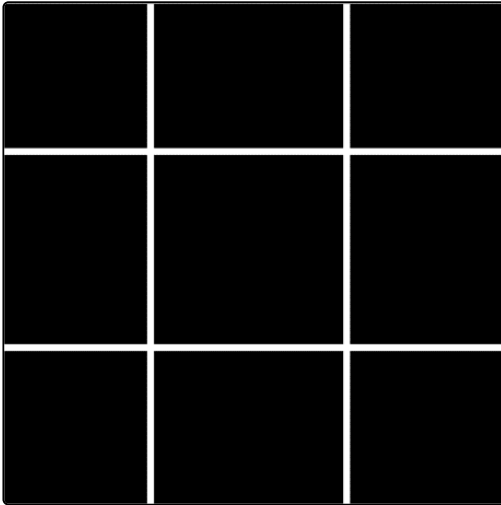
# Linhas Verticais (imagem, (xi,yi), (xf,yf), (color), (thickness))
cv2.line(img, (0,150), (512,150), (255,255,255), 5)
cv2.line(img, (0,350), (512,350), (255,255,255), 5)

cv2.imshow('tic-tac-toe', img)
cv2.waitKey(0) & 0xFF
```



```
cv2.destroyAllWindows()
```

Resultado:



2.2 RETÂNGULO

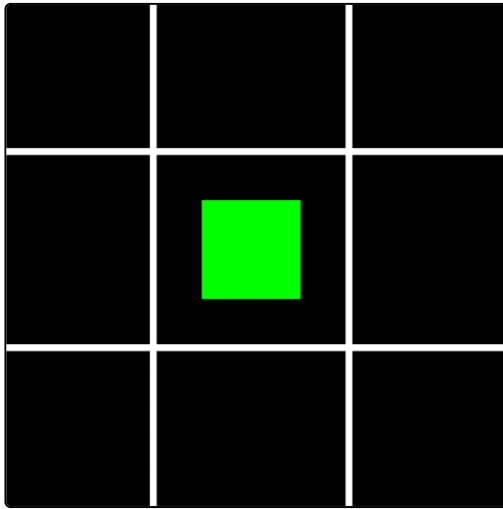
cv2.rectangle() é usado para desenhar um retângulo, passando o **canto superior esquerdo** e o **canto inferior direito**.

- Faremos um retângulo verde no centro da imagem que já estávamos desenhando.
- Lembrete: -1 é preenchimento total da geometria.

```
# Retângulo (imagem, (xi,yi), (xf,yf), (color), (thickness))
cv2.rectangle(img, (200,200), (300,300), (0,255,0), -1)

cv2.imshow('tic-tac-toe', img)
cv2.waitKey(0) & 0xFF
cv2.destroyAllWindows()
```

Resultado:

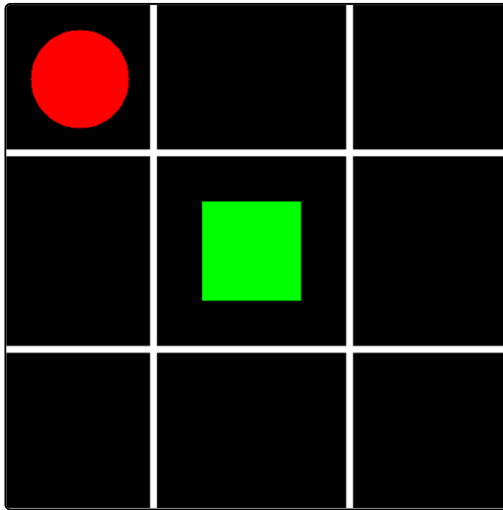


2.3 CÍRCULO

cv2.circle() é usado para desenhar um círculo, deve-se informar as **coordenadas centrais e o raio**.
Desenharemos um círculo vermelho no canto superior esquerdo.

```
# Círculo (imagem, (xc,yc), (raio), (color), (thickness))
cv2.circle(img, (75,75), 50, (0,0,255), -1)
cv2.ellipse(img, (430,430), (60,20), 0, 0, 360, 360, -1)
cv2.imshow("tic-tac-toe", img)
cv2.waitKey(0) & 0xFF
cv2.destroyAllWindows()
```

Resultado:



2.4 TEXTO

Para escrever textos em imagens precisamos especificar:

- Imagem, texto a ser escrito.
- Posição (coordenadas onde o texto deve ser escrito).
- Tipo de fonte.
- Tamanho da fonte.
- Cor, espessura da linha usada, tipo de linha.

Usamos a função **cv2.putText()**.

```
# Tipo de fonte
font = cv2.FONT_HERSHEY_SIMPLEX

# Texto a ser escrito
texto = "Tabuleiro maneiro"

# Escrevendo um texto (imagem, texto, (xi,yi), tipodefonte, tamanhodafonte, color, thickness, lineType)
cv2.putText(img, texto, (50,500), font, 1.5, (191,0,191), 4, cv2.LINE_AA)
```

```
cv2.imshow("tic-tac-toe", img)  
cv2.waitKey(0) & 0xFF  
cv2.destroyAllWindows()
```

Resultado:



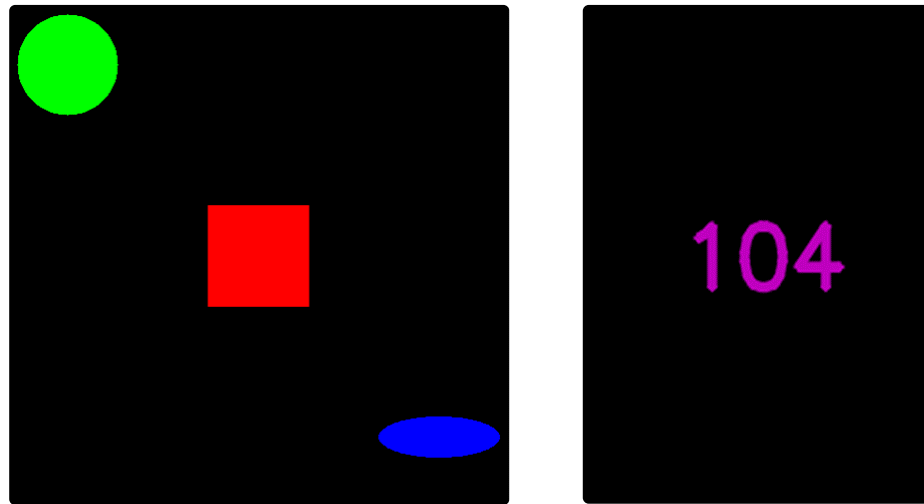
2.5

EXERCÍCIO

Desenvolva uma imagem preta, com três formas geométricas (círculo, retângulo, elipse) na diagonal, em que ao pressionar qualquer tecla irá sair da imagem e mostrar outra imagem com o valor da tecla (verificar a tabela), e manterá a imagem no visor por 1 segundo.

Tempo estimado: 10 minutos.

OUTPUT



Resultado

```
import cv2
import numpy as np

img = np.zeros((500,500,3), np.uint8)

img1 = cv2.circle(img, (60,60), 50, (0,255,0), -1)
img2 = cv2.rectangle(img, (200,200), (300,300), (0,0,255), -1)
img3 = cv2.ellipse(img, (430,430), (60,20), 0, 0, 360, 360, -1)

# Mostra a imagem e transfoma cv2.waitKey(0) em uma variável
cv2.imshow('imagem', img)
x = cv2.waitKey(0)
cv2.destroyAllWindows()

# Transforma o valor da tecla em uma string
k = str(x)

# Cria outra imagem
```

```
img4 = np.zeros((500,500,3), np.uint8)

font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img4, k, (50,500), font, 1.5, (191,0,191), 4, cv2.LINE_AA)

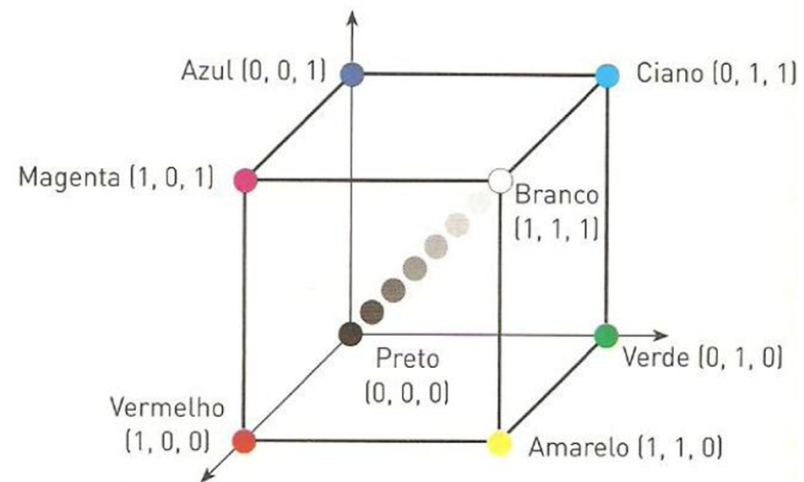
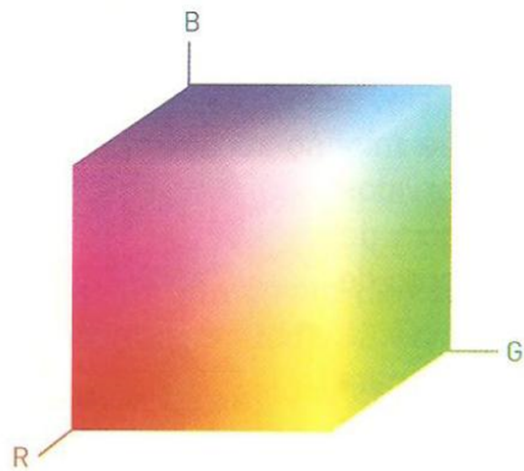
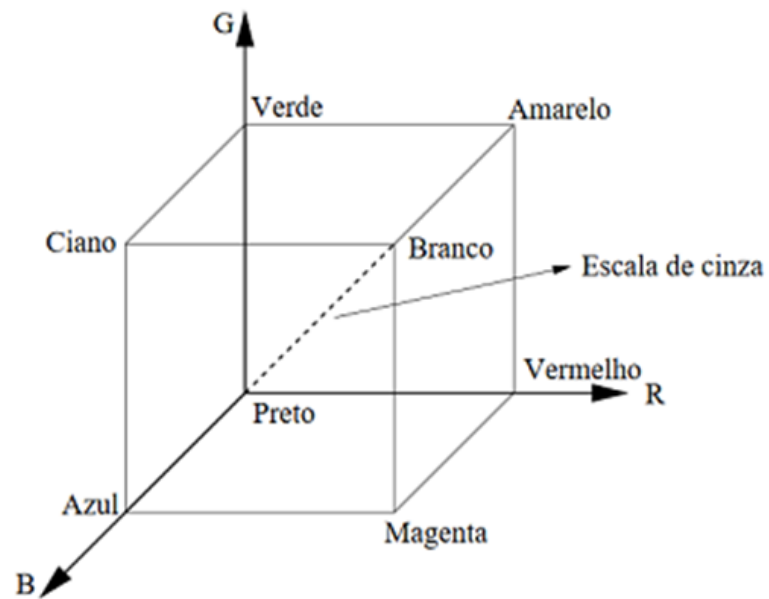
# Mostra a segunda imagem
cv2.imshow('imagem2', img4)
cv2.waitKey(1000)
cv2.destroyAllWindows()
```

2.6

Transformação em espaço de cores

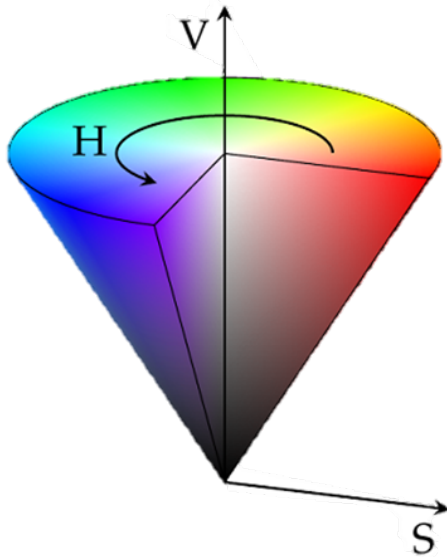
2.6.1 RGB/BGR:

- Um espaço de cores é uma organização específica de cores, um modelo matemático usado para descrever cada cor.
- Um dos mais conhecidos e utilizados é o RGB, sendo **R= red (vermelho)**, **G= green (verde)**, **B= blue (azul)**.
- Com a mistura dessas cores, é possível recriar as cores que percebemos no mundo real. Na biblioteca OpenCV, usa-se o modelo BGR, onde **o vermelho se torna o último valor**.
- Este modelo de cores é baseado em um sistema de coordenadas cartesianas, em que o espaço de cores é um cubo, como mostrado a seguir:



2.6.2 HSV:

- O modelo HSV é definido pelos parâmetros **Matiz (H, hue)**, **Saturação(S, saturation)**, **Luminância ou Brilho(V, value)**.
- **Luminância**: será medida ao longo do eixo vertical, o qual passa pelo centro do cone. É a intensidade luminosa, determina o quão brilhante é uma luz (se mede com base em uma escala de preto para branco);
- **Matiz**: estão representados na parte superior do cone. É o comprimento de onda dominante da cor. Usada para dar um nome a uma cor;
- **Saturação**: é a medida ao longo do eixo horizontal. Mede a pureza relativa da cor ou quantidade de luz branca misturada com matiz.

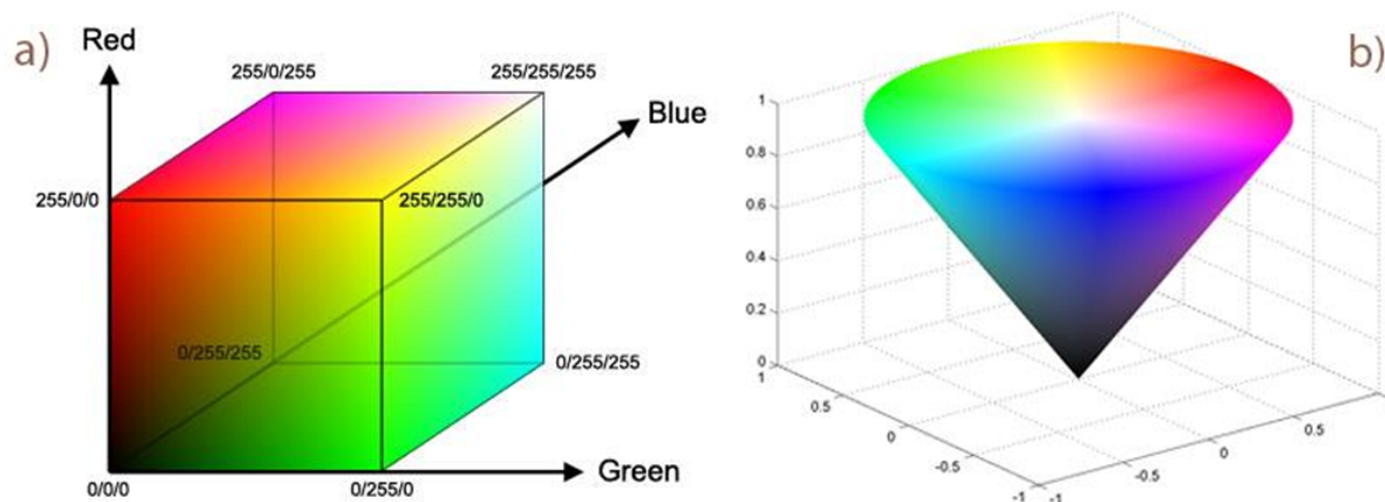


2.7

Mudança de espaço de cor

2.7.1 BGR → HSV

- Para realizar a conversão de cores entre os espaços de cores usamos `cv2.cvtColor(imagem, flag)`. O parâmetro **flag** determina o tipo de conversão a ser feita.
- Assim, se quisermos uma conversão do tipo BGR → HSV utilizamos a flag `cv2.COLOR_BGR2HSV`.



Exemplo

```
import cv2
image = cv2.imread('dog.jpg')
image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

cv2.imshow('Dog_HSV', image_hsv)
cv2.imshow('Original', image)

cv2.waitKey(0) & 0xFF
cv2.destroyAllWindows()
```

Resultado:



2.7.2 BGR → CINZA

- Para os tons de cinza, utilizaremos a mesma função apenas modificando a **flag** que agora será **cv2.COLOR_BGR2GRAY**.

Exemplo

```
import cv2
image = cv2.imread('dog.jpg')
image_cinza = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

cv2.imshow('Dog_Gray', image_cinza)
cv2.imshow('Original', image)

cv2.waitKey(0) & 0xFF
cv2.destroyAllWindows()
```

Resultado:



3 Eventos do Mouse

Utiliza-se a função `cv2.EVENT_(FUNÇÃO)`

Função Mouse call-back `cv2.setMouseCallback()`.

- Parâmetros como **nome da janela** e onde o **evento irá ocorrer**.

```
import cv2

def funcao(event, x, y, flags, param):
    ...

cv2.setMouseCallback('janela', funcao)
```

Eventos no mouse podem ser :

- **RBUTTONDOWN/ LBUTTONDOWN/ MBUTTONDOWN** – Botões para baixo.
- **RBUTTONDBLCLK/ LBUTTONDBLCLK/ MBUTTONDBLCLK** – Dois cliques nos botões.
- **RBUTTONUP/ LBUTTONUP/ MBUTTONUP** – Botões para cima (Após serem pressionados).
- **FLAG_ALTKEY/ FLAG_CTRLKEY/ FLAG_SHIFTKEY**
- **FLAG_LBUTTON/ FLAG_RBUTTON/ FLAG_MBUTTON** – Botões pressionados.
- **MOUSEWHEEL/ MOUSEWHEEL/ MOUSEMOVE** - Movimento do mouse e do scroll.

Exemplo:

Programa que ao clicar na tela é mudada a cor do plano de fundo:

```
import cv2
import numpy as np
import random

def funcao(event, x, y, flags, param):
    global a,b,c
    if event == cv2.EVENT_LBUTTONDOWN:
        a = random.randint(0,255)
```

```
b = random.randint(0,255)
c = random.randint(0,255)

a = 0
b = 0
c = 0

cv2.namedWindow('janela')
cv2.setMouseCallback('janela', funcao)
while(True):
    img = np.full((512,512,3), (a,b,c), np.uint8)
    cv2.imshow('janela', img)
    if cv2.waitKey(10) & 0xFF == 27:
        break
cv2.destroyAllWindows()
```

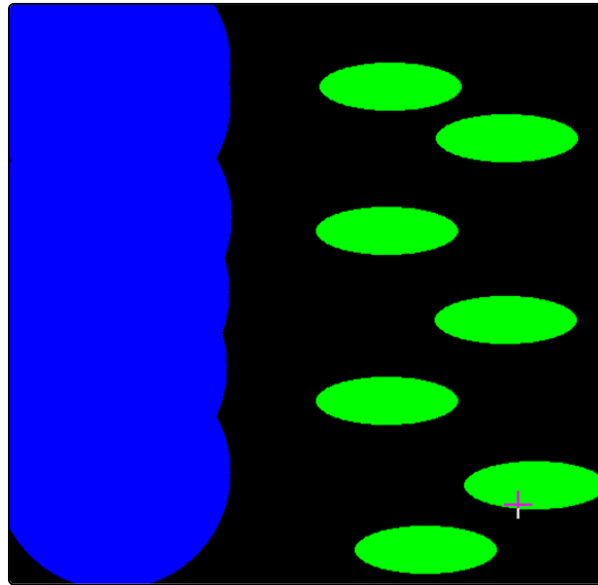
3.1

EXERCÍCIO

Crie uma imagem que ao movimentar o Scroll irá desenhar um círculo azul, ao realizar um Double click no lado esquerdo cria elipse verde, e ao pressionar 'ESC' sairá da imagem.

Tempo estimado: 20 minutos

OUTPUT



Resultado

```
import cv2
import numpy as np

# Mouse callback function
def draw_circle(event,x,y,flags,param):
    if event == cv2.EVENT_MOUSEWHEEL:
        cv2.circle(img, (x,y), 100, (255,0,0), -1)
    elif event == cv2.EVENT_LBUTTONDOWN:
        cv2.ellipse(img, (x,y), (60,20), 0, 0, 360, (0, 255, 0), -1)

# Create a black image, a window and bind the function to window
img = np.zeros((512,512,3), np.uint8)
cv2.namedWindow('image')
cv2.setMouseCallback('image', draw_circle)

while(1):
    cv2.imshow('image', img)
```

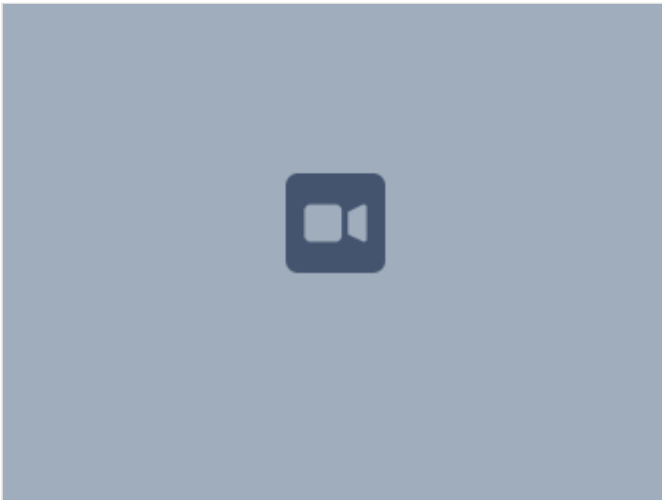
```
if cv2.waitKey(20) & 0xFF == 27:  
    break  
cv2.destroyAllWindows()
```

3.2 EXERCÍCIO

1. Faça um programa que cria uma imagem com fundo preto.
2. Na imagem deve aparecer as palavras “red”, “blue”, “green”.
3. Ao apertar as teclas ‘R’, ‘G’, e ‘B’ respectivamente os valores de cada cor aumentam em 10;
4. Ao clicar com o botão esquerdo do mouse é feito um círculo com os valores de cor definidos;
5. Ao apertar a tecla ‘X’ a imagem é fechada;

Tempo estimado: 40 minutos.

OUTPUT



Resultado

```
import cv2
import numpy as np

mode = True
red = 0
blue = 0
green = 0
font = cv2.FONT_HERSHEY_SIMPLEX

def draw_circle(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        cv2.circle(img, (x,y), 20, (blue,green,red), -1)

img = np.zeros((512,512,3), np.uint8)
cv2.namedWindow('image')
cv2.setMouseCallback('image', draw_circle)

while(1):
    cv2.putText(img, 'red= '+str(red), (10,30), font, 1, (255,255,255), 2)
    cv2.putText(img, 'blue= '+str(blue), (10,80), font, 1, (255,255,255), 2)
    cv2.putText(img, 'green= '+str(green), (10,130), font, 1, (255,255,255), 2)
    cv2.imshow('image', img)
    k = cv2.waitKey(20) & 0xFF
    if k == ord('r'):
        red+=10
        cv2.circle(img, (140,30), 50, (0,0,0), -1)
        if red > 255:
            red=0
    if k == ord('b'):
        blue+=10
        cv2.circle(img, (140,80), 50, (0,0,0), -1)
        if blue > 255:
            blue=0
    if k == ord('g'):
        green+=10
        cv2.circle(img, (160,130), 50, (0,0,0), -1)
```



```
    if green > 255:  
        green=0  
    if k == ord('x'):  
        break  
cv2.destroyAllWindows()
```

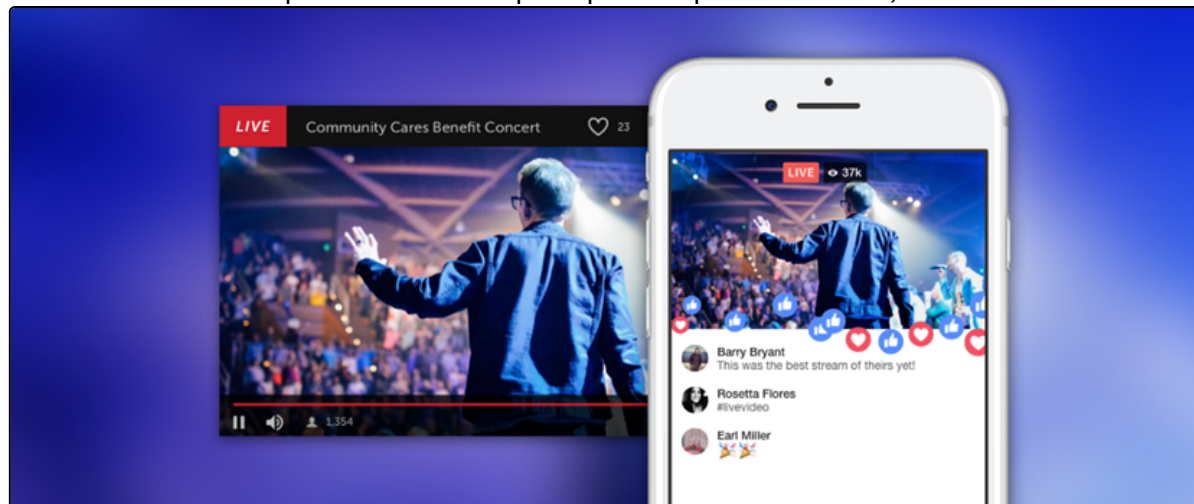
4

VÍDEOS

Um vídeo é um conjunto de imagens distribuídas no tempo. Cada uma dessas imagens é chamada de **frame**.

Para capturar um vídeo live stream com uma câmera, OpenCV oferece uma interface bem simples, mas é preciso criar um objeto do tipo **VideoCapture()**. O argumento pode ser o índice da câmera ou o nome do arquivo de vídeo.

Índice da câmera é apenas um número que especifica qual câmera usar, sendo o **default zero 0**.



Como o valor é True sempre que há um frame, podemos checar se o vídeo chegou ao fim de acordo com o valor retornado (um loop while, por exemplo).

Para visualizar o vídeo também utilizamos **cv2.imshow()**.

Ao final devemos soltar a captura com **cap.release()** e fechar as janelas abertas.

```
import cv2

cap = cv2.VideoCapture('video_car.mp4') # Armazenar o vídeo
ret, frame = cap.read() # Retorna bool, e o frame (True se houver um frame)

while(True):
    ret, frame = cap.read()
    if ret == False:
        break
```

```
cv2.imshow('Corrida', frame)
if cv2.waitKey(1) == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Algumas vezes, **cap** pode não inicializar a captura. Nesse caso, o código retornará um erro. Podemos checar se a captura foi inicializada ou não com **cap.isOpened()**. Se retornar True é porque foi inicializada. Caso contrário, usamos **cap.open()**.

No exemplo abaixo, vamos ler um vídeo salvo no computador. Ao apertar a tecla "q", a janela se fechará.

```
import cv2

cap = cv2.VideoCapture('video_car.avi')

while(cap.isOpened()):
    ret, frame = cap.read()
    if(cv2.waitKey(1) == ord('q') or ret == False):
        break
    cv2.imshow('frame', frame)

cap.release()
cv2.destroyAllWindows()
```

Após capturar e fazer a leitura da frame do vídeo, podemos também salvá-lo.

Diferente do que ocorre com imagens, aqui precisamos criar um objeto do tipo **VideoWriter()**.

O objeto recebe o nome do arquivo, o código **FourCC**, o número de frames por segundo (fps) e o tamanho dos frames.

- **FourCC** é um código que especifica o codec do vídeo.
- **Codec** é um programa que permite comprimir e descomprimir um vídeo digital.
- Os mais usados são **'XVID'** para formato .avi e **'MJPG'** para formato .mp4.

Vamos usar **VideoWriter_fourcc()** para passar o codec.

```
import cv2

cap = cv2.VideoCapture('video_car.avi')

fourcc = cv2.VideoWriter_fourcc(*'XVID')
output = cv2.VideoWriter('novo_video.avi', fourcc, 20.0, (640,480))

while(cap.isOpened()):
    ret, frame = cap.read()
    if ret == True:
        frame = cv2.flip(frame, 0)
        output.write(frame)

        cv2.imshow('frame', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
        else:
            break

cap.release()
output.release()
cv2.destroyAllWindows()
```

4.1

EXERCÍCIO

Faça um vídeo de duração de **10 segundos**.

A cada meio segundo uma forma geométrica muda de posição para uma posição aleatória.

Use 30 fps.

Tempo estimado: 45 minutos.

OUTPUT



Resultado

```
import cv2
import numpy as np

fourcc = cv2.VideoWriter_fourcc(*'XVID')
output = cv2.VideoWriter('exercicio.avi', fourcc, 30.0, (640, 480))
back_color = (140, 100, 80)
color = (0,0,0)

for _ in range(20):
    x = np.random.randint(0,640)
    y = np.random.randint(0,480)

    for _ in range(15):
        frame = np.full((480,640,3), back_color, dtype = np.uint8)
        circ = cv2.circle(frame, (x,y), 50, color, -1)
        output.write(frame)
        cv2.imshow('video', frame)
        cv2.waitKey(1)

output.release()
cv2.destroyAllWindows()
```

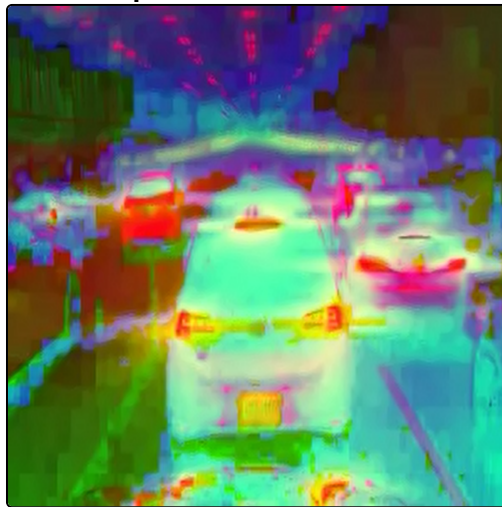
4.2

EXERCÍCIO

Abra o vídeo carro.avi e realize as seguintes modificações:

- Modifique o tamanho para 300x300.
- Converta as cores para o espaço HSV.
- Salve o vídeo com o nome “carro_psicodélicos.avi”.

Tempo estimado: 30 minutos



Resultado

```
import cv2

cap = cv2.VideoCapture('video_car.avi')

fourcc = cv2.VideoWriter_fourcc(*'XVID')
```

```
output = cv2.VideoWriter('carro_psicodélico.avi', fourcc, 30.0, (300,300))

while(cap.isOpened()):
    ret, frame = cap.read()
    if ret == True:
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        frame2 = cv2.resize(frame, (300,300))
        output.write(frame2)
        cv2.imshow('frame', frame2)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    else:
        break

cap.release()
output.release()
cv2.destroyAllWindows()
```